

1. Capa de Servicios: API REST y Controladores

La aplicación se comunica mediante microservicios orquestados, donde el Frontend (Vanilla JS) consume una API REST centralizada.

Endpoints Principales y Lógica de Negocio

- **Gestión de Identidad y Perfil:**
 - **GET /usuarios/perfil:** Intercambia el JWT (almacenado en cookie **HttpOnly**) por el estado del usuario (**AppState**). Devuelve objetos normalizados de inventario, habilidades equipadas y configuración de avatar.
 - **POST /auth/login:** Realiza la autenticación contra el backend. Al ser exitosa, inicializa la sesión en el navegador y permite la carga del ecosistema de microservicios.
- **Telemetría y Rankings:**
 - **GET /usuarios/ranking/global:** Consulta agregada que suma créditos y estadísticas de todos los usuarios activos para construir el "Salón de la Fama".
 - **POST /usuarios/guardar-partida:** Endpoint transaccional. Recibe el JSON de fin de partida (puntuación, aciertos, precisión) y actualiza las métricas en la tabla **partidas** y la entidad **usuarios** (estadísticas acumuladas).
 - **GET /usuarios/estadisticas/{id}:** Endpoint diseñado para la visualización pública de perfiles, recuperando KPIs de rendimiento (mejor roscó, mejor contrarreloj, porcentaje precisión).
- **Generación Dinámica y Economía:**
 - **GET /incidencias/game/rosco-ia?tema={tema}:** Lógica de servidor que delega la generación de contenido a Google Gemini mediante ingeniería de prompts forzando un formato **separado por |**. Incluye sistema de *Fallback* local si la API externa falla.
 - **POST /usuarios/buy/{idItemParam}:** Gestiona la concurrencia en la compra. Verifica fondos en el **AppState** y ejecuta una transacción JPA para descontar créditos e insertar en la tabla **inventario**.

2. Motor de Tiempo Real: WebSockets y STOMP

El modo multijugador (**Versus**) utiliza un modelo de comunicación *Full-Duplex* para reducir la latencia de red en las carreras mecanográficas.

- **Configuración del Broker:**
 - Se utiliza **WebSocketMessageBrokerConfigurer**. Se define **/topic** como el prefijo para los mensajes que el servidor *envía* (broadcast) a los clientes suscritos y **/app** como el prefijo para los mensajes que los clientes *envían* al servidor.
 - El endpoint **/ws-game** permite la conexión mediante **SockJS**, garantizando *fallbacks* de conexión si el navegador del cliente no soporta WebSockets nativos.
- **Ciclo de Vida de una Sala (Versus):**
 - **Host:** Al crear sala, genera un código aleatorio (caracteres **ABC...234**) y publica la creación.

- **Invitado:** Se conecta al endpoint de la sala y realiza el *Handshake*.
- **Sincronización:** Una vez establecida la conexión, el servidor actúa como orquestador, retransmitiendo el evento **BATTLE_START** para inicializar el contador de 3 segundos en ambos clientes simultáneamente.
- **Resiliencia:** Si ocurre una desconexión abrupta (**SessionDisconnectEvent**), el servidor libera los recursos del socket pero mantiene el estado de la partida para que el jugador restante pueda finalizar y reclamar su victoria.

3. Capa de Observabilidad: Programación Orientada a Aspectos (AOP)

El sistema utiliza AOP para desacoplar el registro de logs de la lógica de negocio (Principio de Responsabilidad Única).

- **Definición de Aspectos:**
 - **@Pointcut("within(com.jorge.sprintdef.services..*)"):** Define que todos los métodos dentro de la capa de servicios son sujetos de auditoría.
- **Captura de Excepciones (@AfterThrowing):**
 - Interceptor automático: Cuando un servicio lanza una excepción (**NotFoundException**, **DuplicateException**, etc.), el aspecto captura el **JoinPoint** (método y clase exacta), extrae el mensaje de error y lo persiste en la tabla **incidencias** de la base de datos MySQL.
 - Este sistema garantiza que, aunque un servicio falle, la aplicación no se bloquee y el usuario no reciba una traza de error técnica en el frontend, sino una respuesta controlada.

4. Arquitectura de Despliegue: API Gateway

El sistema está desplegado bajo un diseño de microservicios, lo que requiere un punto de entrada único.

- **Enrutamiento:** **GatewayRoutesConfig** utiliza **RouteLocatorBuilder** para delegar peticiones:
 - **/auth/, /usuarios/, /roles/** -> Dirigido al **usuarios-service**.
 - **/incidencias/** -> Dirigido al **incidencias-service**.
- **CORS y Seguridad:** El Gateway centraliza la gestión de las cabeceras **Access-Control-Allow-Origin** y **Access-Control-Allow-Credentials**, evitando configuraciones redundantes y permitiendo una comunicación segura entre un frontend desacoplado y múltiples microservicios.